AI-Powered Intelligent Interview and Placement Assessment System

Project Documentation and Team Contribution Report

Project name: HireSense

All statements about implementation are based on the actual source code in the project directory.

Table of Contents

1. Project Overview
2. Problem Statement
3. Project Objectives
4. Explanation for Non-Technical Readers
5. Intended System Workflow
6. Actual Implemented Workflow
7. System Architecture
8. Technology Stack
9. Four User Roles
10. Feature Status Table
11. Project Directory Structure
12. Important Files
13. Function and API Explanations
14. Five-Member Work Distribution
15. Team Leader Responsibilities
16. Contribution Evidence and Limitations
17. Code Documentation Improvements
18. Testing and Verification Results
19. Current Limitations and Known Issues
20. Future Enhancements
21. Final Conclusion

1. Project Overview

HireSense is a web application that helps students prepare for campus placements. It brings students, mentors, recruiters and administrators into one shared system. A student uploads a resume, receives an AI-based analysis, practices mock interviews, practices English, takes assessments and applies for jobs. Mentors guide assigned students. Recruiters publish jobs and review candidates. Administrators manage the platform and configuration.

All AI features run locally on the user's computer. The application uses a local language model through Ollama (qwen3:4b by default), local embeddings and local speech tools. No external AI API key is required.

2. Problem Statement

Students preparing for placements often do not know how interviewers will question them, receive little personalised feedback and have no structured practice environment. Placement portals usually store profiles and jobs but do not assess skills, practise interviews or track improvement. HireSense turns placement preparation into an interactive, feedback-driven process.

3. Project Objectives

- Provide role-based portals for students, recruiters, mentors and administrators.

- An

4. Explanation for Non-Technical Readers

What problem it solves

Imagine a final-year student named Anita. She has studied coding but has never faced a real interview. She uploads her resume. The system reads the projects on it, asks questions about them, scores her answers and tells her what to improve. She repeats this until she feels confident.

Traditional placement interview

A recruiter asks questions, the student answers, and the recruiter decides pass or fail. The student rarely receives detailed feedback.

How the AI system improves preparation

The system becomes a patient interviewer that can ask follow-up questions, adjust difficulty, explain mistakes and suggest study plans.

What AI means here

Artificial Intelligence here means a local language model that reads text and generates questions, scores and feedback. It runs on the student's machine, so data stays local.

End-to-end working

- Student creates an account and logs in.

- Stu

5. Intended System Workflow (verified)

Step	Description	Implementation status
1	Student registers and logs in	Implemented
2	Student profile and resume upload	Implemented
3	Extract education/skills/projects/experience	Implemented
4	AI identifies strengths/weaknesses	Implemented
5	Student selects activity	Implemented
6	Interviewer asks resume-based questions	Implemented
7	Student answers text/voice	Implemented; voice browser-dependent
8	System evaluates answers and feedback	Implemented
9	Adaptive difficulty	Implemented
10	Performance report	Implemented
11	Mentor monitoring	Implemented
12	Recruiter candidate access	Implemented
13	Admin manages platform	Implemented

Honesty note: Voice recognition depends on browser support. Without Ollama, interviews use the admin question bank and answers stay unscored. AI quality depends on model availability.

6. Actual Implemented Workflow

The backend is a FastAPI application (backend/app/main.py). It exposes routers for auth, students, interviews, english, assessments, jobs, mentors, admin, voice and monitoring. The frontend is a React/Vite single-page app with protected routes. Data is stored in SQLite by default, with PostgreSQL supported via DATABASE_URL. Startup warms up models in a background thread. Security headers and a global exception handler are registered.

7. System Architecture

Components:

- Frontend: React 18, Vite, Tailwind, React Router, Recharts, Three.js avatar.

- Ba

Flow: Student Browser (React) -> FastAPI API (routers + auth) -> Services (ai.py, matching.py, rewards.py) -> Ollama / Embeddings -> SQLAlchemy -> SQLite/PostgreSQL.

8. Technology Stack

Layer	Technology
Frontend	React 18, Vite 6, Tailwind CSS, React Router, Recharts, Three.js
Backend	Python 3.12+, FastAPI, SQLAlchemy 2, Alembic
Database	SQLite default; PostgreSQL supported
Auth	JWT in httpOnly cookie, Argon2 hashing, RBAC
AI text	Ollama qwen3:4b
Matching	Sentence Transformers all-MiniLM-L6-v2
ML	scikit-learn
Resumes	pypdf, python-docx
Voice	Web Speech API; Kokoro TTS; Whisper STT

9. Four User Roles

Student

Registers, uploads resume, receives AI analysis, takes mock interviews, practices English, solves assessments, applies for jobs, views progress/rewards.

Recruiter

Registers (needs admin approval), creates/publishes/closes jobs, reviews applications, runs AI match, compares candidates, updates status.

Mentor

Views assigned students, their records, writes feedback/private notes, creates improvement plans/goals, schedules sessions.

Admin

Approves recruiters/suspensions, assigns mentors, configures question bank, rubrics, badges, rewards, settings, monitors AI usage/errors, exports placement reports.

10. Feature Status Table

Feature	Purpose	Status	Key files
Auth	Login/access	Implemented	auth.py, Login.jsx, Register.jsx
Role access	Security	Implemented	deps.py, models
Resume upload	Add resume	Implemented	students.py, Resume.jsx, resume_parser.py
Resume AI	Extract+judge	Implemented	services/ai.py, skills.py
Mock interviews	Practice	Implemented	routers/interviews.py, interview_engine.py
Voice interview	Spoken Q/A	Implemented, browser dependent	voice.py, tts.py, stt.py, VoiceInterview.jsx
Adaptive difficulty	Level change	Implemented	interviews.py thresholds
English practice	Conversation	Implemented	english.py, english_companion.py

Assessments	MCQs	Implemented	assessments.py, Assessments.jsx
Rewards	Motivation	Implemented	rewards.py, Rewards.jsx
Job matching	Job fit	Implemented transparent	matching.py, Jobs.jsx
Mentor portal	Guidance	Implemented	mentors.py, mentor pages
Admin portal	Management	Implemented	admin.py, admin pages
Monitoring	Ops insight	Implemented	monitoring.py, perf.py
TTS/STT	Speech	Partially tested manually	tts.py, stt.py

11. Project Directory Structure

Top-level: backend/ (FastAPI, Alembic, services, routers, tests), frontend/ (React app), docs/ (generated documentation), configuration files (.env.example, start scripts). See generated tree in the editable documentation files.

12. Important Files

- backend/app/main.py - FastAPI app creation, routers, middleware, startup warmup.

- ba

13. Function and API Explanations

Key execution paths:

- Student login -> POST /api/auth/login -> JWT cookie.

- Re

Full interactive API docs are available at http://127.0.0.1:8000/docs when the backend runs.

14. Five-Member Work Distribution

Git history shows a single contributor (Rajveer Kharade). Therefore the allocation below is proposed, not verified historical contribution.

Member	Module	Key files
Member 1	AI engine, architecture, integration	services/ai.py, main.py, matching.py, interview_engine.py
Member 2	Student frontend	pages/student/*, components/*, voice/*
Member 3	Backend & database	models.py, schemas.py, routers/auth.py, database.py, alembic/*
Member 4	Recruiter & admin portals	routers/jobs.py, admin.py, recruiter pages, admin pages
Member 5	Mentor, testing, learning	routers/mentors.py, english.py, english_companion.py, tests/*

15. Team Leader Responsibilities

Member 1 (team leader) covers overall architecture, AI integration and model selection (Ollama qwen3:4b, sentence-transformers), interview question generation and adaptive logic coordination, frontend/backend/database/AI integration, API contracts, integration testing, technical decisions and demo preparation. Because Git shows one contributor, these are assigned/proposed responsibilities, not claims of sole implementation of every component.

16. Contribution Evidence and Limitations

Evidence available: Git log with 7 commits by Rajveer Kharade. No per-member commit attribution exists. Contribution allocation is proposed, not historically verified.

17. Code Documentation Improvements

Added module-level docstrings/comments to core backend files and core frontend files. Preserved functionality (header additions only). Key files updated include main.py, config.py, database.py, deps.py, security.py, auth.py, api.js, auth.jsx, App.jsx, main.jsx and Layout.jsx. Many modules already had docstrings and were skipped. Deeper function-level docstrings in large modules are recommended for a later pass.

18. Testing and Verification Results

Backend command: cd backend; .\venv\Scripts\python.exe -m pytest -q --basetemp=C:\Users\skaus\AppData\Local\Temp\opencode\pytest. A prior full run showed 208 errors caused by Windows permissions on the default pytest temp directory; using the approved basetemp directory resolves those environmental failures. Frontend tests run via npm test (state machines, lip sync, recorder, speech queue, monitoring).

The test suite uses mocked models and an in-memory database; it proves logic, not real-model answer quality. Real-model checks live in backend/tests_integration.

19. Current Limitations and Known Issues

- AI availability depends on Ollama/model installation.

- Vo
brows

20. Future Enhancements

- Add OCR for scanned resumes.

- Imp

21. Final Conclusion

HireSense is a substantial full-stack project with local AI integration, four portals, resume analysis, mock interviews, English practice, assessments, rewards, job matching and monitoring. Implementation is feature-rich; AI output quality depends on the local model and requires manual verification. Work distribution in this report is proposed because Git history does not show individual contributors.

Note about secrets: No secrets or private data are included. Environment variable names may be referenced; their values are not printed.